



UNIVERSITE PARIS CITE

Conception Détaillée

Projet L2G1

Outil de traçabilité automatisé

Date : Février 2026

Maîtrise d'ouvrage (Réfèrent) :

M. Jeremy MEYNADIER

Équipe de développement :

Mme Emilie HUANG

Mme Lamia ARRAHMANE

Mme Amel KHELOUL

Mme Yseult HELLE RIGLET

Table des matières

1	Introduction	2
1.1	Objet du document.....	2
1.2	Portée du document	2
2	Architecture générale	2
2.1	Schéma global de l'architecture	3
2.2	Découpage en modules fonctionnels	3
3	Module 1 : Import et Parsing	3
3.1	Description du module.....	4
3.2	Rappel des objectifs et besoins liés	4
3.3	Interactions (Composants utilisés et utilisateurs)	4
3.4	Interfaces et visibilités	4
3.5	Classes, structures de données et méthodes	4
3.5.1	Classe : ExtracteurFichier.....	4
3.5.2	Structure de données associée : ExigenceExtraite	5
3.5.3	Gestion des anomalies (Exceptions personnalisées)	6
3.6	Schéma d'activité : Flux d'import d'un document	7
4	Module 2 : Moteur de Traçabilité	7
4.1	Description du module.....	8
4.2	Rappel des objectifs et besoins liés	8
4.3	Interactions (Composants utilisés et utilisateurs)	9
4.4	Interfaces et visibilités	11
4.5	Classes, structures de données et méthodes	13
5	Module 3 : Interface graphique	18
5.1	Objectifs fonctionnels du module	19
5.2	Architecture du Module 3.....	19
5.3	Structure de données et classes principales.....	20
5.3.1	Classe : MainWindow	20
5.3.2	Classe : OngletRapport.....	22
5.3.3	Classe : VueGrapheTracabilite.....	22
5.3.4	Classe : GestionnaireFiltresGUI.....	23
5.4	Algorithme d'affichage du graphe.....	24
6	Matrice de traçabilité	24
6.1	États représentés dans la matrice.....	25
6.2	Construction dynamique de la matrice.....	25
6.3	Interaction utilisateur avec la matrice.....	25
6.4	Filtrage des données dans la matrice	26
6.5	Performance, gestion des erreurs et validation.....	26
7	Glossaire	26
8	Index	30
9	Référence	33
10	Suivi des modifications	34

1 Introduction

1.1 Objet du document

Le présent document constitue la conception détaillée de l'outil de traçabilité développé par le groupe L2G1. Il a pour but de décrire avec précision l'architecture logicielle de la solution, en détaillant le rôle, les responsabilités et le fonctionnement interne de chaque module avant le début de la phase d'implémentation. Il détaille la conception en respectant les exigences définies dans le Cahier des Charges [REF-01].

1.2 Portée du document

L'application développée est un outil de bureau destiné à vérifier la traçabilité des exigences au sein de projets d'ingénierie. Elle permet d'importer des documents de spécifications de différents niveaux, d'en extraire les identifiants uniques, et de générer un rapport de couverture calculant les taux de traçabilité globaux et intermédiaires tout en mettant en évidence les anomalies.

2 Architecture générale

2.1 Schéma global de l'architecture

L'outil est structuré en plusieurs modules indépendants afin de séparer les responsabilités et faciliter la maintenance. Chaque module gère une partie spécifique du traitement : l'import et l'extraction des données, l'analyse de traçabilité, et la visualisation graphique des résultats. Cette architecture modulaire permet d'optimiser le développement, les tests et l'évolution future de l'outil.

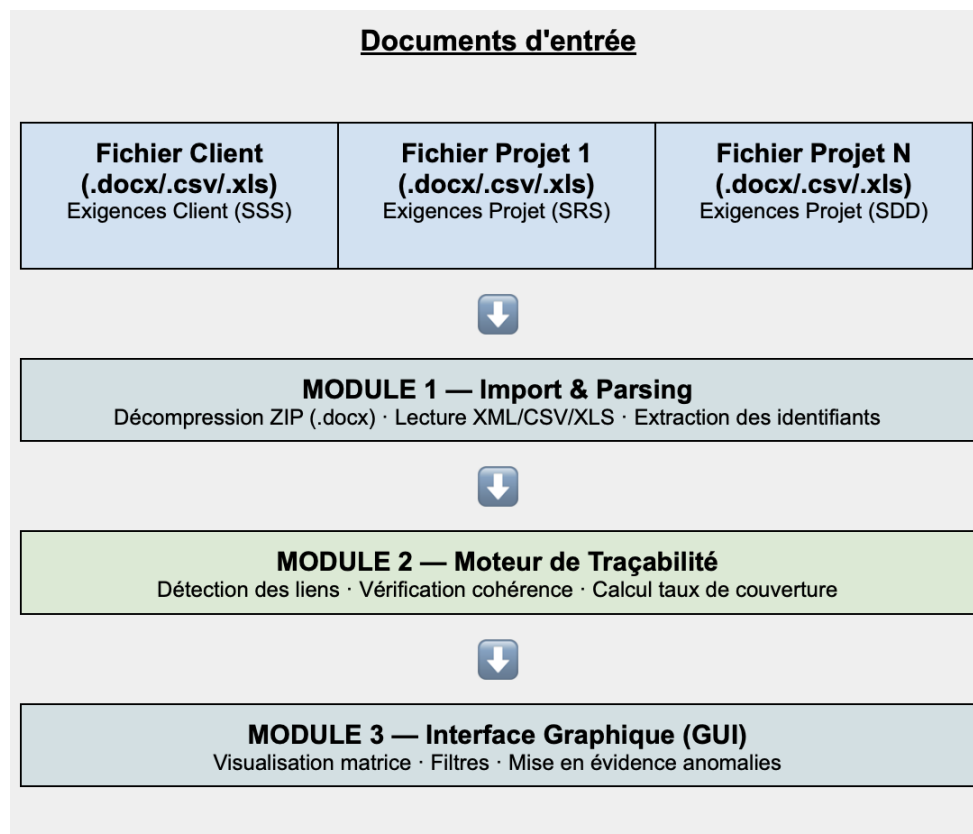


FIGURE 1 – Architecture modulaire du système de traçabilité

2.2 Découpage en modules fonctionnels

Le système est divisé en 3 modules strictement indépendants :

1. **Module 1 (Import et Parsing)** : S'occupe de l'accès aux fichiers physiques et de l'extraction des données brutes (Frontière système).
2. **Module 2 (Moteur de Traçabilité)** : Cœur algorithmique de l'application. Il reçoit les données brutes, applique les règles métier et génère des statistiques.
3. **Module 3 (Interface Graphique)** : Gère les interactions avec l'utilisateur. Il capture les actions (clics, choix de fichiers) et affiche visuellement les résultats calculés par le Module 2.

3 Module 1 : Import et Parsing

3.1 Description du module

Le module d'extraction est le composant « frontière » entre le système de fichiers de l'ordinateur et le cœur logique de l'application. Il est chargé d'accéder aux fichiers physiques en lecture seule, d'en extraire le texte brut, et de filtrer ce texte pour n'isoler que les informations pertinentes (les identifiants des exigences).

3.2 Rappel des objectifs et besoins liés

Objectif : Transformer des données non structurées (fichiers texte ou csv) en données structurées exploitables par le programme.

Besoins liés : Capacité du logiciel à charger des fichiers de type SSS, SRS et SDD.

3.3 Interactions (Composants utilisés et utilisateurs)

Composants utilisés par ce module :

Les bibliothèques standard (STL) du C++ :

- `<fstream>` pour la manipulation des flux de fichiers
- `<regex>` pour le filtrage par expressions régulières [REF-04]
- `<string>` pour la manipulation de texte
- `<vector>` comme structure de données de stockage
- `<stdexcept>` comme classe de base pour les exceptions personnalisées

Composants utilisant ce module :

- **Le Module 2 (Moteur de Traçabilité) :** transmet le chemin du fichier sélectionné par l'utilisateur ainsi que le motif Regex au Module 1, puis reçoit les éventuelles exceptions en cas d'erreur.
- **Le Module 3 (Interface Graphique) :** transmet le chemin du fichier sélectionné par l'utilisateur ainsi que le motif Regex au Module 1, puis reçoit les éventuelles exceptions en cas d'erreur.

3.4 Interfaces et visibilité

Le module expose une interface volontairement restreinte pour protéger la manipulation des fichiers et garantir l'intégrité du processus d'extraction.

Visibilité	Élément	Description
Public (+)	<code>ExtracteurFichier(chemin, motif, motifTracabilite)</code>	Constructeur. Initialise l'extracteur avec le chemin, le motif regex principal et un motif de traçabilité optionnel.
Public (+)	<code>extraire()</code>	Méthode principale. Détecte le format et retourne le vecteur d' <code>ExigenceExtraite</code> .
Public (+)	<code>getNombreExigencesDetectees()</code>	Retourne le nombre d'exigences détectées.
Public (+)	<code>detecterExtension()</code>	Retourne l'extension du fichier source en minuscules.
Privé (-)	<code>verifierExistence()</code>	Vérifie que le fichier existe. Lève <code>FileNotFoundException</code> sinon.
Privé (-)	<code>lireCSV(ext)</code>	Lit un fichier CSV ou XLS. Retourne les paires (numéroLigne, ligne brute).
Privé (-)	<code>lireDocx()</code>	Décompresse et parse word/document.xml. Retourne les paires (numéroLigne, paragraphe).
Privé (-)	<code>lireXlsx()</code>	Lit un fichier .xlsx via les shared strings et les feuilles de calcul.

Privé (-)	lireDoc()	Convertit .doc en .docx via LibreOffice, puis appelle lireDocx().
Privé (-)	lireXls()	Convertit .xls en .xlsx via LibreOffice, puis appelle lireXlsx().
Privé (-)	filtrerLignes(lignes, format)	Applique le filtre regex. Retourne les ExigenceExtraite avec métadonnées.
Privé (-)	parserBlocsDocx(paragraphes)	Regroupe les paragraphes en blocs et extrait tracabilité, cibles, alloueA, etat, methodeVerification.
Privé (-)	decouper(ligne)	Découpe une chaîne 'val1, val2' en vecteur de tokens nettoyés.
Privé (-)	chargerSharedStrings(archive)	Charge la table des chaînes partagées d'un fichier .xlsx.

3.5 Classes, structures de données et méthodes

3.5.1 Classe : ExtracteurFichier

Afin d'assurer les fonctionnalités de ce module, nous utiliserons une classe principale nommée ExtracteurFichier.

Son rôle est de regrouper l'ensemble du processus de récupération des données brutes. Instanciée pour chaque document à analyser, elle gère le flux de lecture et applique les filtres nécessaires pour isoler les identifiants d'exigences. La lecture des fichiers compressés ou spécifiques s'appuie sur les bibliothèques *zlib* [REF-05] et *TinyXML2* [REF-06].

Attributs :

Visibilité	Attribut	Type et description
private	cheminSource	std::string — Chemin absolu ou relatif du fichier source.
private	motifRegex	std::regex — Règle de reconnaissance des identifiants d'exigences.
private	motifTracabiliteRegex	std::regex — Motif secondaire pour les champs de traçabilité (optionnel).
private	listeIdentifiants	std::vector<std::string> — Identifiants validés après filtrage.
private	nbDetectees	int — Nombre d'exigences détectées lors de la dernière extraction.

Méthodes et Algorithmes :

Visib.	Signature	Description
public	ExtracteurFichier(std::string chemin, std::string motif)	Constructeur. Initialise l'objet avec le chemin du fichier et le motif regex. Affecte chemin à cheminSource et convertit motif en motifRegex.
public	std::vector<std::string> extraire()	Méthode principale. Déclenche le processus complet d'extraction. Lève FileNotFoundException ou FileFormatException en cas d'erreur.
private	void verifierExistence() const	Vérifie l'existence et l'accessibilité du fichier source. Lève FileNotFoundException si introuvable.
private	std::vector<std::pair<int, std::string>> lireCSV(const std::string& ext) const	Lit un fichier CSV ou XLS et retourne les paires (numéro de ligne, contenu brut) pour traitement.
private	std::vector<std::pair<int, std::string>> lireDocx() const	Lit un fichier Word (.docx) paragraphe par paragraphe et retourne les paires (numéro, contenu texte).
private	std::vector<ExigenceExtraite> filtrerLignes(...) const	Applique le filtre regex sur chaque ligne brute. Retourne les lignes correspondantes sous forme de struct ExigenceExtraite enrichies du numéro de ligne, de la source et du format.

3.5.2 Structure de données associée : ExigenceExtraite

Struct C++ retournée par filtrerLignes(). Elle encapsule une exigence validée avec toutes les méta-données nécessaires au Moteur de Traçabilité (Module 2).

Champ	Type	Description
identifiant	std::string	Identifiant extrait par la regex (ex. EXIGENCE_CLIENT_VOIE_F01).
contenuBrut / description	std::string	Texte descriptif de l'exigence tel que lu dans le fichier.
fichierSource	std::string	Chemin du fichier d'origine — propagé jusqu'à l'objet Exigence pour les messages de liens brisés.
formatSource	std::string	Extension détectée automatiquement (csv, docx, xlsx, doc, xls).
numeroLigne	int	Numéro de ligne/paragraphe dans le fichier source.
tracabilite	std::vector<std::string>	IDs SSS couverts par cette SRS (colonne 2 CSV / ligne Traçabilité : docx).
cibles	std::vector<std::string>	Cibles logicielles/matérielles (ligne Cible : docx).
necessaireA	std::vector<std::string>	Projets/clients concernés (ligne Nécessaire à : docx).
alloueA	std::vector<std::string>	Projets auxquels la SSS est allouée (ligne Alloué à : docx).
developpe	bool	true si « Développé » figure dans le bloc docx.
etat	std::string	État de l'exigence (ex. : En cours, Validé...).
methodeVerification	std::string	Méthode de vérification associée.
detecterExtension()	std::string	Méthode membre — déduit l'extension depuis fichierSource.

3.5.3 Gestion des anomalies (Exceptions personnalisées)

Pour garantir la robustesse du module d'extraction et permettre une remontée d'erreur propre vers l'interface utilisateur (Module 3) sans faire planter le programme, des classes d'exceptions spécifiques sont définies (héritant de `std::runtime_error`). La conformité de ces remontées d'erreurs est vérifiée lors des phases de tests de robustesse décrites dans le Cahier de Recette [REF-03].

Classe	Héritage	Constructeur	Rôle
FileNotFoundException	std::runtime_error	FileNotFoundException(std::string chemin)	Signalé lorsque le chemin fourni ne permet pas d'ouvrir le fichier (introuvable, supprimé, droits insuffisants). Génère le message : « <i>Fichier introuvable : <chemin></i> ».
FileFormatException	std::runtime_error	FileFormatException(std::string detail)	Signalé si le fichier existe mais ne correspond pas au format attendu (ex : binaire au lieu de texte). Génère le message : « <i>Format non supporté : .pdf</i> ».

3.6 Schéma d'activité : Flux d'import d'un document

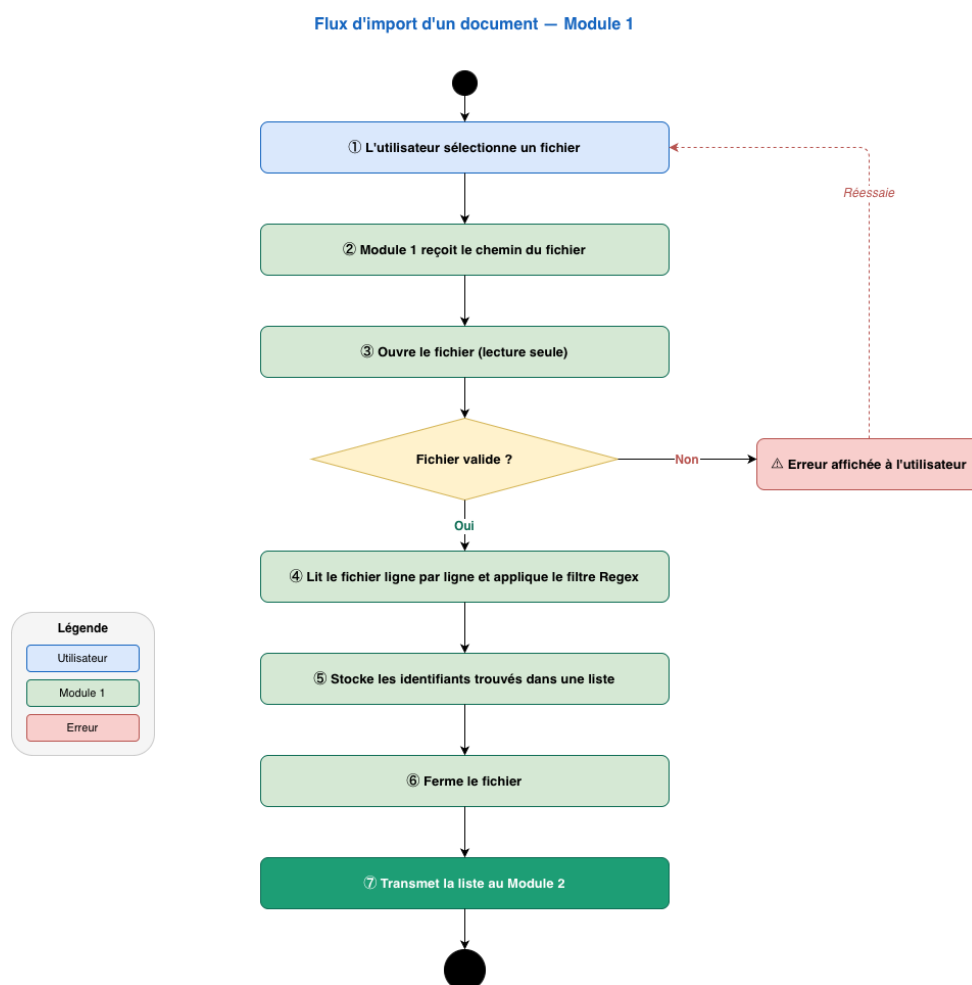


FIGURE 2 – Schéma d'activité — Import et traitement d'un document

4 Module 2 : Moteur de Traçabilité

4.1 Description du module

Le module 2 constitue le cœur algorithmique du système. Il est responsable de l'analyse automatique des liens de traçabilité entre les exigences issues des documents SSS (exigences client), SRS (exigences projet) et SDD (exigences de conception). À partir des données extraites par le module 1, le moteur :

- établit les liens valides entre exigences de différents niveaux,
- détecte les exigences non couvertes,
- identifie les liens brisés (références vers des identifiants inexistant),
- calcule les taux de traçabilité intermédiaires et globaux,
- génère une structure de données synthétisant les résultats pour le module 3 (interface graphique).

Ce module ne gère aucun affichage ; il réalise uniquement les traitements logiques et statistiques.

4.2 Rappel des objectifs et besoins liés

Vérification de la couverture des exigences

Besoin	Description détaillée
Couverture SSS → SRS	Chaque exigence SSS doit être reprise par au moins une exigence SRS. Si ce n'est pas le cas, l'exigence SSS est marquée comme NonCouverte.
Couverture SRS → SDD	Chaque exigence SRS doit être reprise par au moins une exigence SDD. Si ce n'est pas le cas, l'exigence SRS est marquée comme NonCouverte.

Détection des incohérences documentaires

Incohérence détectée	Description détaillée
Référence vers identifiant inexistant	Une SRS référence un identifiant SSS qui n'existe pas dans le vecteur SSS, ou une SDD référence un identifiant SRS inexistant. Ces cas sont signalés dans liensBrises.
Intégrité de la chaîne de traçabilité	Le moteur vérifie que la chaîne complète SSS → SRS → SDD est cohérente de bout en bout, sans rupture ni référence fantôme.

Calcul d'indicateurs quantitatifs

Indicateur	Calcul	Rôle
Taux SSS → SRS	Nb SSS couvertes / nb total SSS	Mesure quelle proportion des exigences système est couverte par des exigences logicielles.
Taux SRS → SDD	Nb SRS couvertes / nb total SRS	Mesure quelle proportion des exigences logicielles est couverte par des exigences de conception.
Taux global	TauxSSS_SRS × TauxSRS_SDD	Indicateur synthétique de bout en bout.
Taux tracabilité complète	Nb SSS avec chaîne SSS → SRS → SDD complète / nb total SSS	Proportion de SSS dont la chaîne est intégralement couverte jusqu'au niveau SDD.

Production d'un rapport structuré

Contenu du rapport	Type	Rôle
Exigences SSS avec statut	vector<ExigenceSSS>	Permet de voir quelles SSS sont couvertes ou non.
Liens brisés	vector<string>	Liste les identifiants référencés qui n'existent pas, avec le fichier source.
Orphelines	vector<string>	Liste les SRS sans référence SSS et les SDD sans référence SRS, séparées des liens brisés.
Taux SSS-SRS et SRS-SDD	double × 2	Taux intermédiaires affichés séparément.
Taux global	double	Indicateur synthétique unique de la qualité de la traçabilité.
Taux de traçabilité complète	double	Proportion de SSS dont la chaîne SSS-SRS-SDD est intégralement couverte.

4.3 Interactions (Composants utilisés et utilisateurs)

Interaction avec le Module 1 (Importateur)

Donnée reçue	Type	Description détaillée
Vecteur SSS	vector<ExigenceSSS>	Exigences système converties. Chaque objet contient alloueA, etat, methodeVerification, fichierSource.
Vecteur SRS	vector<ExigenceSRS>	Exigences logicielles. Chaque objet contient tracabilite (liens SSS), cibles, necessaireA, fichierSource.
Vecteur SDD	vector<ExigenceSDD>	Exigences de conception. Chaque objet contient referencedSRS (liens SRS) et fichierSource.

Le module 2 ne lit jamais directement les fichiers — il reçoit uniquement les structures déjà construites par le module 1. La séparation des responsabilités est totale.

Interaction avec le Module 3 (Interface graphique)

Donnée transmise	Type	Description détaillée
Statuts des exigences	Via RapportTracabilite	Chaque exigence SSS porte son statut mis à jour (Couverte ou NonCouverte) — le module 3 les affiche dans la matrice.
Liste des liens brisés	vector<string>	Messages d'erreur à afficher comme alertes dans l'interface.
Taux calculés	double × 3	Les trois taux prêts à être affichés en pourcentage dans l'interface.

Le module 3 ne fait qu'afficher — il ne calcule rien. Toute la logique reste dans le module 2.

Utilisateur final

Rôle	Description détaillée
Interaction indirecte	L'utilisateur final ne manipule jamais directement le moteur de traçabilité. Il n'a accès qu'à l'interface graphique du module 3.
Déclenchement de l'analyse	C'est en interagissant avec l'interface (bouton, menu) que l'utilisateur déclenche indirectement l'appel au moteur via le module 3.

Utilisation des threads

Thread	Rôle	Responsabilités détaillées
Thread principal	Interaction utilisateur	Gère l'import des fichiers SSS, SRS et SDD. Gère l'affichage des résultats dans le module 3. Reste toujours réactif car il ne fait pas les calculs lourds.
Thread d'analyse (arrière-plan)	Exécution du moteur	Lance analyserTracabilite() et calculerTaux(). Réalise les liaisons entre exigences, détecte les liens brisés et calcule les taux, sans jamais bloquer l'interface.
Signal de fin	Communication inter-threads	À la fin de l'analyse, le thread d'arrière-plan envoie un signal au thread principal pour lui dire que les résultats sont prêts et que l'interface peut être mise à jour.

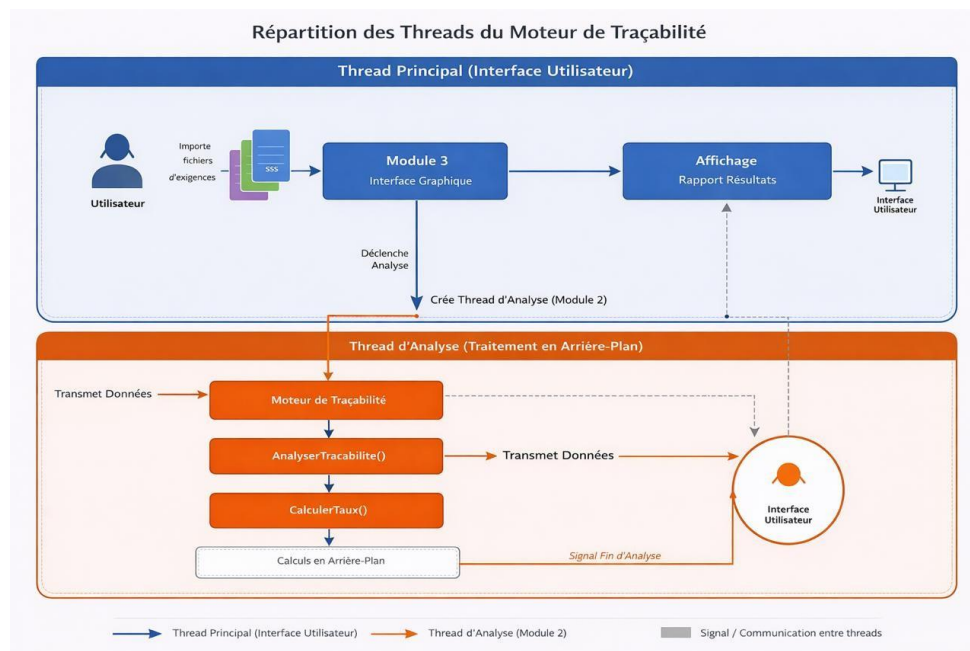


FIGURE 3 – Architecture d'utilisation des threads (Module 2)

Diagramme d'activité détaillé du module 2 — Moteur de Traçabilité :

Le diagramme d'activité ci-dessous détaille le fonctionnement interne du module 2, en précisant le flux d'exécution depuis la réception des données issues du module 1 jusqu'à la transmission du rapport structuré au module 3, en passant par l'ensemble des étapes d'analyse réalisées par le moteur de traçabilité dans son thread dédié. Ce diagramme constitue le détail de la colonne « Module 2 » du diagramme d'activité général présenté dans le cahier de conception globale.

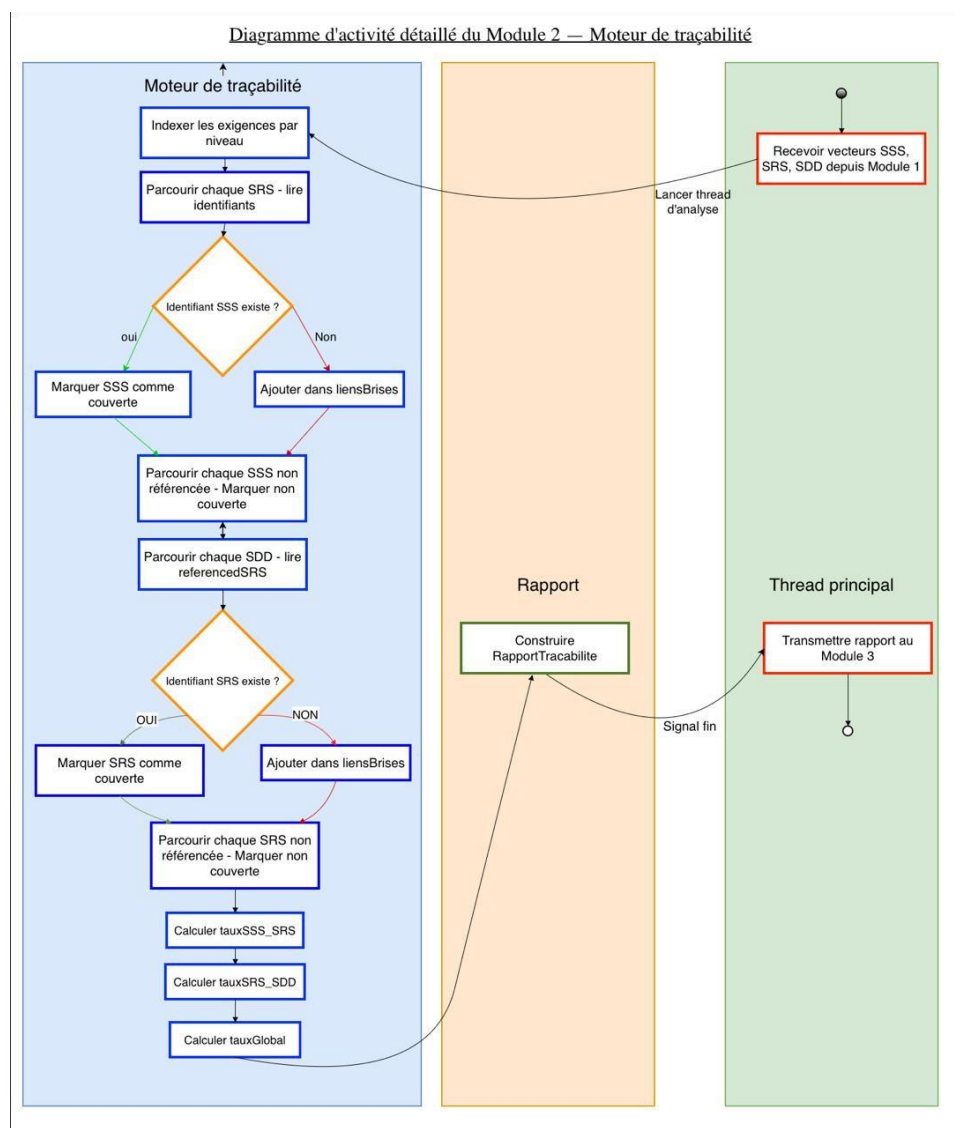


FIGURE 4 – Diagramme d'activité détaillé du module 2

4.4 Interfaces et visibilité

1. Visibilité des classes

Classe	Visibilité	Rôle détaillé
ConvertisseurExigences	Publique	Transforme les ExigenceExtraite en objets typés ExigenceSSS/SRS/SDD selon des préfixes configurables par fichier (splitVirgule, detecterType, estTypeConnu).
Exigence	Interne au moteur	Classe mère abstraite. Contient id, contenu, type, statut, fichierSource.
ExigenceSSS	Interne au moteur	Hérite d'Exigence. Attributs : alloueA, etat, methodeVerification, developpe.
ExigenceSRS	Interne au moteur	Hérite d'Exigence. Attributs : tracabilite, cibles, necessaireA, etat, methodeVerification, developpe.
ExigenceSDD	Interne au moteur	Hérite d'Exigence. Attributs : referencedSRS, developpe.
RapportTracabilite	Publique vers M3	Structure transmise au Module 3 : exigencesSSS, liensBrises, orphelines, tauxSSS_SRS, tauxSRS_SDD, tauxGlobal, tauxTracabiliteComplete.

2. Interface publique de MoteurTracabilite

Méthode	Paramètres	Retour	Rôle détaillé
setDonnees()	3 vecteurs SSS, SRS, SDD	void	Injecte les trois vecteurs d'exigences typées reçus du ConvertisseurExigences.
analyserTracabilite()	Aucun	void	Réinitialise les statuts, vide liensBrises et orphelines, enchaîne analyserSSS_SRS() et analyserSRS_SDD() et calculerTaux().
calculerTaux()	Aucun	void	Calcule tauxSSS_SRS, tauxSRS_SDD, tauxGlobal, tauxTracabiliteComplete et détecte les orphelines.
getTauxSSS_SRS()	Aucun	double	Retourne le taux de couverture SSS→SRS.
getTauxSRS_SDD()	Aucun	double	Retourne le taux de couverture SRS→SDD.
getTauxGlobal()	Aucun	double	Retourne le taux global.
getTauxTracabiliteComplete()	Aucun	double	Retourne la proportion de SSS dont la chaîne SSS→SRS→SDD est complète.
getLiensBrises()	Aucun	vector<string>	Retourne les messages de liens brisés avec nomFichier source.
getOrphelines()	Aucun	vector<string>	Retourne les orphelines (SRS sans SSS ou SDD sans SRS), séparées des liens brisés.
getExigencesSRS()	Aucun	vector<ExigenceSRS>	Accès au vecteur SRS après analyse.
getExigencesSDD()	Aucun	vector<ExigenceSDD>	Accès au vecteur SDD après analyse.
getRapport()	Aucun	RapportTracabilite	Retourne le rapport complet pour le Module 3.

4.5 Classes, structures de données et méthodes

3. Classe Exigence (classe mère)

Attributs :

Attribut	Type	Rôle détaillé	Exemple
id	std::string	Identifiant unique. Clé de recherche principale.	"EXIG_CLIENT_01"
contenu	std::string	Texte brut de l'exigence.	"Le système doit..."
type	TypeExigence	Niveau : SSS, SRS ou SDD.	SSS
statut	Statut	NonAnalyse, Couverte ou NonCouverte.	Couverte
fichierSource	std::string	Chemin du fichier d'origine — utilisé dans les messages de liens brisés et orphelines.	"sss_client.docx"

Énumérations :

Énumération	Valeur	Signification détaillée
TypeExigence	SSS	L'exigence vient du document Spécifications Système — besoin exprimé par le client au niveau système.
TypeExigence	SRS	L'exigence vient du document Spécifications Logiciel — traduction technique d'un besoin client en termes logiciels.
TypeExigence	SDD	L'exigence vient du document Spécifications de Conception — choix technique précis de conception.
Statut	NonAnalyse	Valeur par défaut à la création. Si une exigence reste à ce statut après l'analyse, c'est un bug.
Statut	Couverte	Le moteur a trouvé au moins une exigence du niveau suivant qui référence cette exigence.
Statut	NonCouverte	Le moteur n'a trouvé aucune exigence du niveau suivant qui référence cette exigence. Anomalie.

Méthodes :

Méthode	Retour	Rôle détaillé
getId()	std::string	Retourne l'identifiant de l'exigence. Utilisé massivement par le moteur pour comparer les identifiants. Déclaré const.
getStatut()	Statut	Retourne le statut actuel de l'exigence. Utilisé par calculerTaux() pour compter les exigences couvertes. Déclaré const.
setStatut()	void	Seule méthode qui modifie l'état d'une exigence. Appelée par le moteur pendant analyserTracabilite().

4. Classe ExigenceSSS

Attribut	Type	Rôle détaillé	Exemple
developpe	bool	Indique si cette exigence système a été développée et détaillée. Une exigence non développée peut signifier qu'elle est encore en cours de rédaction.	true / false
alloueA	vector<string>	Liste des projets ou sous-systèmes auxquels cette exigence est allouée. Une même exigence peut concerner plusieurs projets.	["ProjetA", "ProjetB"]

5. Classe ExigenceSRS

Attributs spécifiques :

Attribut	Type	Rôle détaillé	Exemple
tracabilite	vector<string>	Attribut le plus important. Contient la liste des identifiants SSS que cette SRS couvre. C'est ce vecteur que le moteur lit pour établir les liens SSS→SRS.	["SSS-REQ-1", "SSS-REQ-3"]
cibles	vector<string>	Plateformes ou environnements techniques sur lesquels cette exigence s'applique.	["Windows", "Linux"]
necessaireA	vector<string>	Clients ou utilisateurs pour lesquels cette exigence est nécessaire.	["ClientA", "ClientB"]

Méthode importante :

Méthode	Retour	Rôle détaillé
getTracabilite()	vector<string>	Retourne la liste des identifiants SSS référencés par cette SRS. Méthode clé de toute la traçabilité SSS→SRS — le moteur l'appelle sur chaque SRS pour savoir quelles SSS elle couvre.

6. Classe ExigenceSDD

Attribut	Type	Rôle détaillé	Exemple
referencedSRS	vector<string>	Contient la liste des identifiants SRS que cette SDD prend en compte. C'est ce vecteur que le moteur lit pour établir les liens SRS→SDD.	["SRS-REQ-1", "SRS-REQ-2"]

7. Attributs privés de MoteurTracabilite

Champ	Type	Rôle détaillé
exigencesSSS	vector<ExigenceSSS>	Copie des SSS avec statuts mis à jour. Le module 3 les affiche dans la matrice.
liensBrises	vector<string>	Messages d'erreur de liens brisés avec nomFichier source.
orphelines	vector<string>	Messages d'orphelines (SRS sans tracabilite ou SDD sans referencedSRS), séparées des liens brisés.
tauxSSS_SRS	double	Taux de couverture SSS→SRS prêt à être affiché (multiplié par 100 pour le pourcentage).
tauxSRS_SDD	double	Taux de couverture SRS→SDD.
tauxGlobal	double	Indicateur synthétique — taux SSS→SRS × taux SRS→SDD.
tauxTracabiliteComplete	double	Proportion de SSS dont la chaîne SSS→SRS→SDD est intégralement couverte.

8. Méthode analyserTracabilite()

⇒ Vérifié par SC-MOT-01, SC-MOT-02, SC-MOT-03, SC-MOT-04.

Analyse SSS → SRS :

Étape	Action	Détail complet
1	Parcourir chaque exigence SRS	On itère sur exigencesSRS. Pour chaque SRS on lit sa liste de références SSS via getTracabilite().
2	Lire les identifiants référencés	Pour chaque identifiant dans tracabilite de la SRS courante, on cherche s'il existe dans exigencesSSS.
3a	Identifiant trouvé	On appelle setStatut(Couverte) sur la SSS correspondante.
3b	Identifiant non trouvé	Lien brisé. On ajoute un message dans liensBrises : « Lien brisé : SRS-REQ-X référence SSS-REQ-Y qui n'existe pas ».
4	Parcourir chaque exigence SSS	Après toutes les SRS, on reparcourt exigencesSSS pour vérifier lesquelles n'ont pas été marquées Couverte.
5	SSS encore à NonAnalyse	Aucune SRS ne l'a référencée — on appelle setStatut(NonCouverte).

Analyse SRS → SDD :

Étape	Action	Détail complet
1	Parcourir chaque exigence SDD	On itère sur exigencesSDD. Pour chaque SDD on lit sa liste de références SRS via getReferencedSRS().
2	Lire les identifiants référencés	Pour chaque identifiant dans referencedSRS de la SDD courante, on cherche s'il existe dans exigencesSRS.
3a	Identifiant trouvé	On appelle setStatut(Couverte) sur la SRS correspondante.
3b	Identifiant non trouvé	Lien brisé — on ajoute un message dans liensBrises.
4	Parcourir chaque exigence SRS	On reparcourt exigencesSRS pour vérifier lesquelles ne sont pas Couverte.
5	SRS encore à NonAnalyse	On appelle setStatut(NonCouverte).

9. Méthode calculerTaux()

⇒ Vérifié par SC-MOT-05.

Étape	Calcul	Cas limite et comportement
1	Compter les SSS Couverte → $\text{tauxSSS_SRS} = \text{nb_couvertes} / \text{nb_total}$	Si exigencesSSS vide → $\text{tauxSSS_SRS} = 0.0$ + avertissement dans liensBrises.
2	$\text{tauxSSS_SRS} = \text{nb_SSS_couvertes} / \text{nb_total_SSS}$	Résultat entre 0.0 et 1.0. Ex. : 3 couvertes sur 4 → 0.75.
3	Compter les SRS Couverte → $\text{tauxSRS_SDD} = \text{nb_couvertes} / \text{nb_total}$	Si exigencesSRS vide → $\text{tauxSRS_SDD} = 0.0$ + avertissement.
4	$\text{tauxSRS_SDD} = \text{nb_SRS_couvertes} / \text{nb_total_SRS}$	Résultat entre 0.0 et 1.0.
5	$\text{tauxGlobal} = \text{tauxSSS_SRS} \times \text{tauxSRS_SDD}$	Ex. : $0.75 \times 0.5 = 0.375$ (37.5%).
6	tauxTracabiliteComplete : pour chaque SSS, vérifier qu'il existe une SRS qui la couvre ET est elle-même Couverte par une SDD.	Résultat entre 0.0 et 1.0.
7	Orphelines SDD : SDD sans referencedSRS → push dans orphelines.	Séparé de liensBrises.
8	Orphelines SRS : SRS sans tracabilite → push dans orphelines.	Les SRS avec lien brisé ne sont pas orphelines — déduplication automatique.

10. Structure RapportTracabilite

Champ	Type	Rôle détaillé
exigencesSSS	vector<ExigenceSSS>	Copie de toutes les exigences SSS avec leurs statuts mis à jour après analyse. Le module 3 les affiche dans la matrice.
liensBrises	vector<string>	Liste complète de tous les messages d'erreur. Le module 3 les affiche comme alertes pour que l'utilisateur puisse corriger les incohérences.
tauxSSS_SRS	double	Taux de couverture SSS→SRS prêt à être affiché. Le module 3 peut le multiplier par 100 pour l'afficher en pourcentage.
tauxSRS_SDD	double	Taux de couverture SRS→SDD prêt à être affiché.
tauxGlobal	double	Indicateur synthétique le plus important — dit en un seul chiffre à quel point la chaîne SSS→SRS→SDD est complète.

11. Diagramme UML des classes du module 2

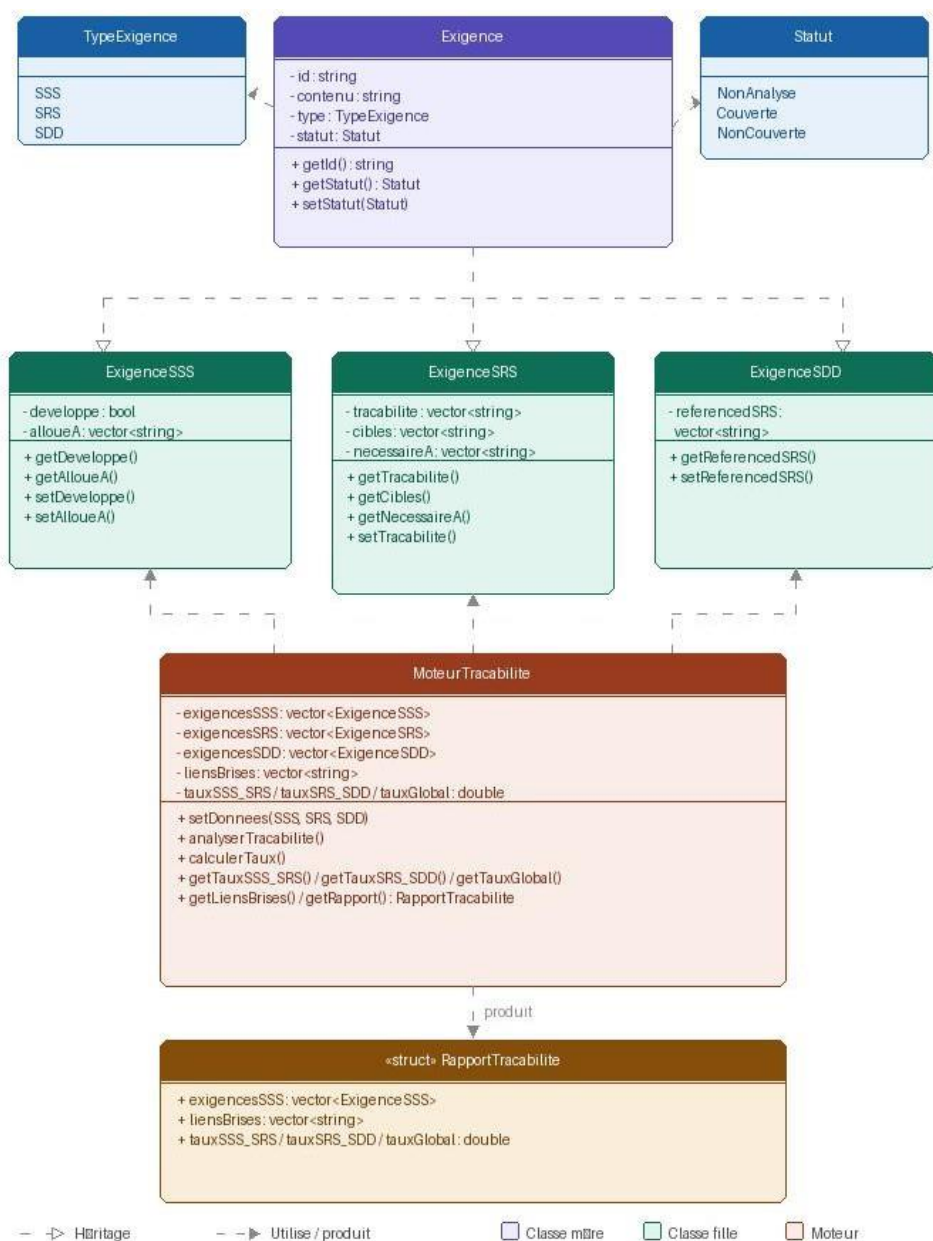


FIGURE 5 – Diagramme UML des classes du module 2

12. Structure d'utilisation des threads

Bibliothèque utilisée :

Bibliothèque	Version minimale	Rôle
<thread>	C++11	Permet de créer et gérer des threads indépendants pour exécuter des tâches en parallèle.

Utilisation des threads dans le projet :

Mécanisme	Outil C++	Rôle détaillé
Création d'un thread	<code>std::thread</code>	Chaque analyse est lancée dans un thread séparé pour ne pas bloquer l'interface. On lui passe la fonction d'analyse comme point d'entrée.
Attente de fin du thread	<code>join()</code>	Garantit que tous les résultats sont complets avant la mise à jour de l'interface.
Protection des données partagées	<code>std::mutex</code>	Protège l'accès aux vecteurs partagés quand plusieurs threads y accèdent simultanément.
Verrouillage automatique	<code>std::lock_guard</code>	Verrouille et déverrouille automatiquement l'accès à une ressource partagée. Plus sûr que la gestion manuelle.

Données partagées à protéger :

Donnée partagée	Type	Risque sans protection
Vecteur des exigences SSS	<code>vector<ExigenceSSS></code>	Deux threads pourraient modifier les statuts en même temps → résultats corrompus.
Vecteur des exigences SRS	<code>vector<ExigenceSRS></code>	Même risque que SSS.
Vecteur des liens brisés	<code>vector<string></code>	Deux threads pourraient écrire simultanément → messages perdus ou mélangés.

Bénéfices de cette approche :

Bénéfice	Explication
Interface réactive	Le thread principal gère l'interface pendant que le thread d'analyse tourne en arrière-plan — l'utilisateur n'a pas l'impression que le programme est bloqué.
Meilleures performances	Les traitements lourds s'exécutent sans interrompre les autres opérations.
Robustesse	Grâce à <code>mutex</code> et <code>lock_guard</code> , les données partagées sont toujours dans un état cohérent.

5 Module 3 : Interface graphique

Le Module 3 est la partie visible du logiciel. Son rôle est d'afficher les résultats calculés par le Module 2 et de permettre à l'utilisateur d'interagir avec les données de traçabilité. Il ne réalise aucun calcul : il reçoit les données et les présente.

5.1 Objectifs fonctionnels du module

Exigence	Fonctionnalité	Description
EX-18	Matrices de traçabilité	Deux tableaux 3 colonnes : Exigence Statut Couverte par — un pour SSS×SRS, un pour SRS×SDD.
EX-19	Code couleur	Vert = couvert, Rouge = non couvert, Orange = lien brisé, Violet = orpheline.
EX-20	Filtrage des résultats	Filtrage par statut, état développé, identifiant (autocomplétion) et type de lien (brisé/orpheline).
EX-21	Taux de couverture	Taux SSS–SRS, SRS–SDD, global et traçabilité complète avec barres de progression.
EX-22	Fonctionnement local	Application entièrement locale, sans connexion réseau.
EX-23	Export multi-format	Export CSV, HTML interactif (filtres JavaScript) et PDF via QPrinter.
EX-24	Écran de démarrage	Fenêtre Accueil avec barre de chargement animée (QTimer 25ms) et bouton Démarrer.

5.2 Architecture du Module 3

Le Module 3 est organisé en quatre classes, chacune représentant un élément concret et visible de l'application.

Classe	Rôle concret
Accueil	Fenêtre de démarrage (splash screen) avec barre de progression animée. Émet demarrerClique() pour ouvrir MainWindow.
MainWindow	Fenêtre principale. Coordonne l'import multi-fichiers SSS/SRS/SDD, la configuration des préfixes par fichier, le lancement de l'analyse et l'affichage des onglets.
OngletRapport	Onglet rapport : taux avec barres, matrices SSS×SRS et SRS×SDD, badges résumé, panneaux liens brisés (orange) et orphelines (violet) cliquables, exports CSV/HTML/PDF.
VueGrapheTracabilite	Onglet graphe : nœuds par fichier (NoeudFichier), liens avec pourcentages (LienFichier), couleurs fixes par type, zoom/pan, vue directe/indirecte.
GestionnaireFiltresGUI	Panneau de filtres : statut, état développé, recherche par mot-clé avec QCompleter. Mode graphe (cache statut/état).

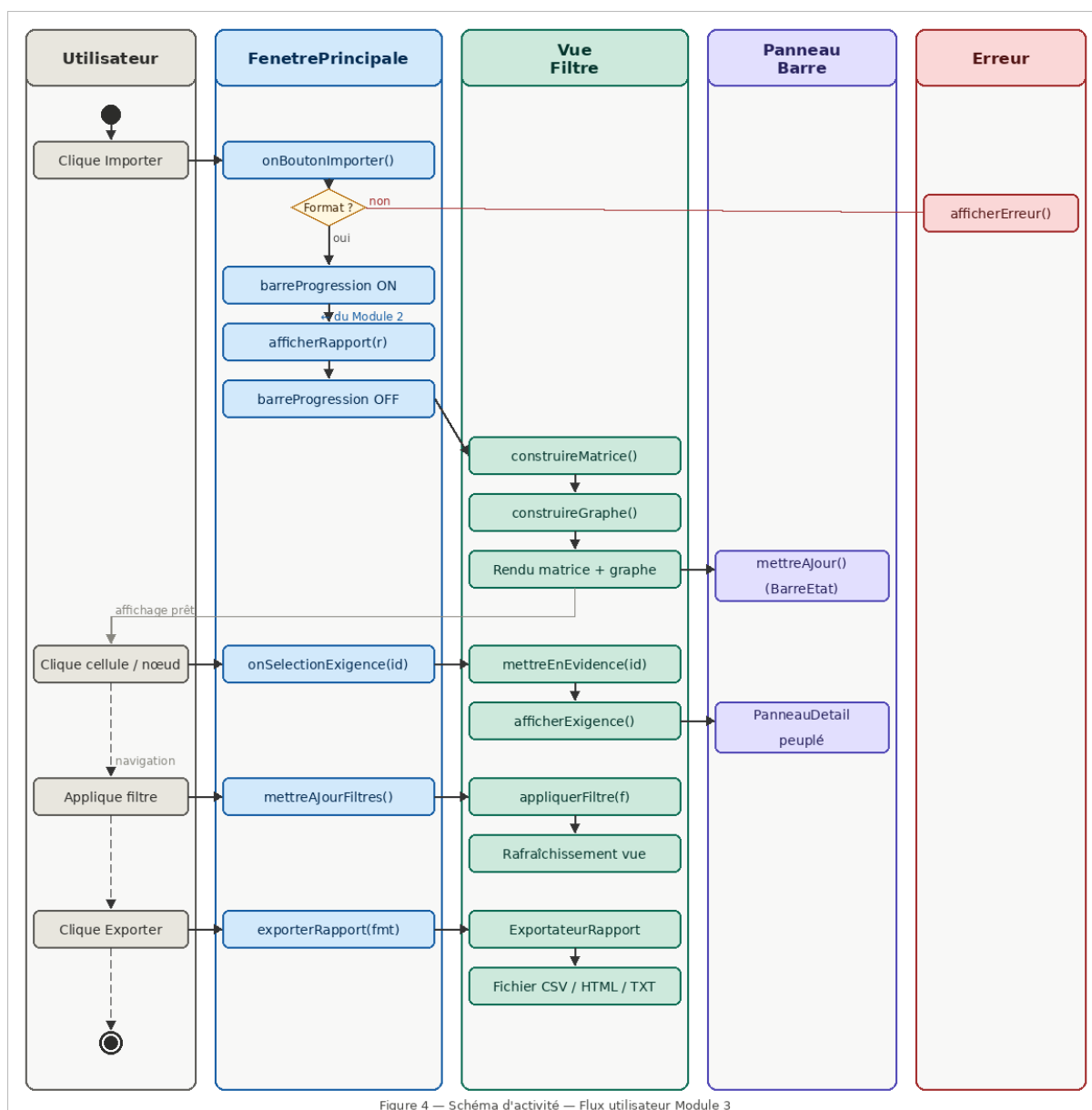


FIGURE 6 – Schéma d'activité — Flux utilisateur Module 3

5.3 Structure de données et classes principales

5.3.1 Classe : Accueil

Fenêtre de démarrage (QWidget sans frame). S'affiche avant MainWindow. Avance automatiquement une barre de progression via QTimer toutes les 25 ms, puis active le bouton Démarrer. Le signal demarrerClique() est connecté à MainWindow::show() dans main.cpp.

Attributs :

Attribut / Méthode	Type	Visib.	Description
barreChargement	QProgressBar*	Privé	Barre animée (0–100, 25 ms/tick).
btnDemarrer	QPushButton*	Privé	Activé à 100%. Émet demarrerClique() au clic.

timer	QTimer*	Privé	Déclenche avancerChargement() toutes les 25 ms.
labelChargement	QLabel*	Privé	Texte animé : Initialisation... →Presque prêt...
demarrerClique()	signal Qt	—	Émis au clic — connecté à MainWindow::show() dans main.cpp.

Méthodes :

5.3.2 Classe : MainWindow

Fenêtre principale (QMainWindow). Coordonne l'import multi-fichiers SSS/SRS/SDD, la configuration des préfixes par fichier, le lancement de l'analyse via ThreadManager et l'affichage des onglets Rapport et Graphe.

Attribut	Type	Visib.	Description
tabWidget	QTabWidget*	Privé	Onglets principaux : Import Rapport Graphe.
cheminsSSS/SRS/SDD	QStringList	Privé	Chemins des fichiers sélectionnés pour chaque niveau.
prefixesParFichierSSS/SRS/SDD	QMap<QString, QLineEdit*>	Privé	Map chemin → champ de saisie du préfixe, un par fichier importé.
terminal	QTextEdit*	Privé	Journal de log coloré.
moteurAnalyse	MoteurTracabilite*	Privé	Instance du moteur allouée à chaque analyse.
threadManager	ThreadManager*	Privé	Gère le thread d'analyse.

Affiche la matrice de traçabilité sous forme de grille colorée. Elle reçoit les données du rapport et colorie chaque case selon le statut de la relation entre deux exigences.

Attributs :

Méthode	Paramètres	Retour	Rôle
parcourirSSS/SRS/SDD()	Aucun	void	Ouvre un QFileDialog multi-sélection et génère les champs de préfixes.
creerChampsPrefixes(...)	QStringList, layout, map	void	Génère un QLineEdit par fichier pour saisir son préfixe de classification.
verifierFichiersSelectionnes()	Aucun	void	Active le bouton Lancer si au moins un fichier est sélectionné par type.
lancerAnalyse()	Aucun	void	Extrait, convertit (préfixes par fichier), injecte dans MoteurTracabilite, lance ThreadManager, appelle afficherRapportComple().
logTerminal(message, couleur)	QString × 2	void	Ajoute un message coloré dans le terminal de log.
reinitialiser()	Aucun	void	Vide toutes les sélections, listes et remet l'interface à zéro.

Méthodes :

5.3.3 Classe : OngletRapport

Onglet Qt affichant le rapport de traçabilité complet produit par le Module 2.

Méthode	Description
afficherRapportComplet(rapport, srs, sdd, orphelines)	Met à jour taux, barres, badges, matrices SSS×SRS et SRS×SDD, panneaux liens brisés et orphelines.
exporterCSV()	Génère un fichier CSV avec taux, matrices et liens brisés.
exporterHTML()	Génère une page HTML interactive avec recherche temps réel et filtres JavaScript, onglets SSS→SRS et SRS→SDD.
exporterPDF()	Export PDF via QTextDocument et QPrinter.
appliquerFiltre(filtre)	Cache les lignes de matrices ne correspondant pas au filtre actif.
eventFilter(obj, event)	Intercepte les clics sur les badges liens brisés et orphelines pour toggle les panneaux.
toggleFiltres()	Affiche/cache le panneau de filtres latéral.

Affiche le graphe des liens de traçabilité. Chaque exigence est un nœud coloré. Chaque lien valide est une arête avec le pourcentage de couverture affiché dessus.

Attributs :

Attribut	Type	Visib.	Description
noeuds	vector<NoeudFichier>	Privé	Un nœud par fichier importé (SSS, SRS ou SDD).
liens	vector<LienFichier>	Privé	Liens de traçabilité inter-fichiers avec pourcentage.
vueIndirecte	bool	Privé	true = vue indirecte (pourcentageIndirect).
niveauZoom	double	Privé	Facteur de zoom (1.0 = normal). Molette souris.
noeudSelectionne	int	Privé	Index du nœud sélectionné au clic.
COULEUR_SSS/SRS/SDD	static QColor	Privé	Couleurs fixes : #1A3A5C / #2E86C1 / #5DADE2.
panneauInfo	QFrame*	Privé	Barre affichant les taux globaux dont tauxTracabiliteComplete.

Méthodes :

Méthode	Paramètres	Retour	Rôle
construireGraphe(rapport, srs, sdd, orphelines)	RapportTracabilite, vectors	void	Crée NoeudFichier et LienFichier. Position SSS gauche, SRS centre, SDD droite.
paintEvent(event)	QPaintEvent*	void	Redessine nœuds colorés par type et liens avec badge de pourcentage.
wheelEvent(event)	QWheelEvent*	void	Zoom avant/arrière à la molette.
mousePressEvent(event)	QMouseEvent*	void	Sélection d'un nœud — affiche le panneau détail.
appliquerFiltre(filtre)	const FiltreGUI&	void	Filtre la visibilité des nœuds.
setVueDirecte() / setVueIndirecte()	Aucun	void	Bascule entre les deux modes d'affichage.
dessinerNoeud(...)	QPainter, NoeudFichier, bool, bool	void	Rectangle coloré avec nom de fichier et taux.
dessinerLien(...)	QPainter, LienFichier, bool	void	Flèche avec badge de pourcentage. Lien brisé en pointillés orange.

5.3.4 Classe : GestionnaireFiltresGUI

Gère les filtres disponibles dans la barre de filtres de l'interface. Quand l'utilisateur modifie un filtre, cette classe met à jour l'affichage sans relancer l'analyse.

Attributs :

Attribut / Méthode	Type / Retour	Visib.	Description
comboStatut	QComboBox*	Privé	Filtre statut : Tous Couvert Non couvert Lien brisé (98) Orpheline (99).
comboEtat	QComboBox*	Privé	Filtre état : Tous Développé Non développé En cours.
champIdentifiant	QLineEdit*	Privé	Champ de recherche avec autocomplétion (QCompleter).

filtreCourant	FiltreGUI	Privé	Structure contenant tous les critères actifs.
modeGraphe	bool	Privé	true = cache statut et état (inutiles dans la vue graphe).
appliquerFiltres()	void	Slot	Construit filtreCourant et émet filtreModifie(filtre).
reinitialiser()	void	Slot public	Remet tous les combos à 0 et émet filtreModifie(vide).
setListeIds(ids)	void	Public	Met à jour la liste d'autocomplétion du QCompleter.
filtreModifie(filtre)	signal	—	Émis après chaque application de filtre.

Méthodes :

Champ FiltreGUI	Type	Valeur par défaut	Description
filtreStatut	Statut	NonAnalyse	Statut à afficher (NonAnalyse = pas de filtre).
filtreIdentifiant	QString	""	Sous-chaîne de l'identifiant à rechercher.
filtreDevEtat	bool	false	true si un filtre état développé est actif.
filtreOrpheline	bool	false	true = afficher seulement les orphelines.
filtreLienBrise	bool	false	true = afficher seulement les liens brisés.
filtreEnCours	bool	false	true = afficher seulement En cours.

5.4 Algorithme d'affichage du graphe

Étape	Action	Résultat visible pour l'utilisateur
1	Récupération des données	MainWindow reçoit le RapportTracabilite du Module 2. La barre de progression disparaît.
2	Création des nœuds et arêtes	VueGrapheTracabilite crée un nœud par exigence (coloré vert/rouge/orange) et une arête par lien avec le pourcentage affiché.
3	Application des filtres	GestionnaireFiltresGUI applique les filtres actifs. Les nœuds hors critères sont masqués.
4	Affichage final	Le graphe et la matrice sont affichés. L'utilisateur peut zoomer, cliquer sur un nœud ou une case, ou modifier les filtres.

6 Matrice de traçabilité

La matrice de traçabilité est la vue principale du Module 3. C'est une grille qui permet de voir d'un seul coup d'œil quelles exigences sont couvertes, lesquelles ne le sont pas, et où se trouvent les liens brisés. Elle est construite automatiquement à partir du RapportTracabilite fourni par le Module 2.

6.1 États représentés dans la matrice

État	Signification concrète	Couleur	Exemple
couverte	L'exigence client (SSS) est reprise dans au moins une exigence projet (SRS). Le lien existe et est valide.	Vert	EXIGENCE_CLIENT_VOIE_1 → EXIGENCE_LOGICIEL_1
non_couverte	Aucune exigence projet ne fait référence à cette exigence client. Un besoin du client n'est pas traité.	Rouge	EXIGENCE_CLIENT_VOIE_2 → (aucune référence SRS)
lien_brise	Une exigence projet référence un identifiant client qui n'existe pas. Erreur de saisie ou identifiant supprimé.	Orange	EXIGENCE_LOGICIEL_3 → EXIGENCE_CLIENT_VOIE_99

6.2 Construction dynamique de la matrice

Étape	Ce qui se passe	Résultat
1	Récupération du rapport	Le Module 3 reçoit le RapportTracabilite du Module 2, contenant toutes les exigences SSS/SRS/SDD avec leurs statuts et la liste des liens brisés.
2	Organisation des exigences	Les identifiants SSS sont placés en lignes, les identifiants SRS en colonnes, triés par ordre alphabétique.
3	Remplissage des cases	Pour chaque paire (SSS, SRS) : lien valide → case verte. Aucun lien → case rouge. Identifiant absent → case orange.
4	Affichage	La grille colorée est affichée. L'utilisateur peut zoomer, cliquer sur une case et appliquer des filtres.

6.3 Interaction utilisateur avec la matrice

Action de l'utilisateur	Méthode déclenchée	Ce qui se passe à l'écran
Clique sur une case	afficherRapportComplet(), exporterCSV(), exporterHTML(), exporterPDF().	Affiche les détails de l'exigence : identifiant, type de document, statut et liste des liens associés.
Zoom avant / arrière	zoomer(facteur)	La grille s'agrandit ou se réduit. Les en-têtes restent toujours visibles.
Clique sur un en-tête de ligne	mettreEnEvidence(id)	Toutes les cases de cette ligne SSS sont mises en évidence.
Clique sur « Exporter »	exporterCSV(chemin)	Une fenêtre s'ouvre pour choisir où sauvegarder. Un fichier CSV est généré avec les résultats filtrés.
Clique sur « Réinitialiser les filtres »	reinitialiser()	Tous les filtres sont remis à zéro et la matrice complète est à nouveau affichée.

6.4 Filtrage des données dans la matrice

Type de filtre	Comment l'utilisateur l'utilise	Effet sur la matrice
Par document	Choisit SSS, SRS, SDD ou Tous dans un menu déroulant.	Seules les lignes ou colonnes du niveau choisi restent visibles.
Par statut	Choisit Couverte, Non couverte, Lien brisé ou Tous dans un menu déroulant.	Seules les cases avec le statut choisi restent visibles. Pratique pour repérer rapidement les anomalies.
Par identifiant	Tape un texte dans le champ de recherche (ex. « VOIE »).	Seules les lignes et colonnes dont l'identifiant contient ce texte restent visibles.

6.5 Performance, gestion des erreurs et validation

Critère	Exigence	Comment c'est géré
Performance	Affichage fluide avec beaucoup d'exigences	Seules les cases modifiées sont redessinées lors d'un changement de filtre. L'objectif est un affichage sans ralentissement visible.
Filtre sans résultat	Filtre trop restrictif	La matrice affiche « Aucun résultat pour ce filtre ». L'application ne plante pas.
Erreur d'export	Impossible d'écrire le fichier	Une boîte de dialogue affiche « Impossible d'écrire le fichier. Vérifiez les droits d'accès. » L'application reste utilisable.
Validation des données	Les données affichées sont-elles correctes ?	Le Module 3 n'invente aucune donnée : il affiche uniquement ce qu'il reçoit du Module 2. Vérifiable via le scénario SC-INT-02 du cahier de recettes.

7 Glossaire

Ce glossaire répertorie l'ensemble des termes techniques, acronymes et classes utilisés dans ce document de conception détaillée. Les entrées sont classées par ordre alphabétique.

Terme	Catégorie	Définition
SSS	Acronyme	System/Subsystem Specification. Document de spécification des exigences client (niveau 1 de la chaîne de traçabilité).
SRS	Acronyme	Software Requirements Specification. Document de spécification des exigences projet (niveau 2 de la chaîne de traçabilité).
SDD	Acronyme	Software Design Document. Document de conception logicielle détaillée (niveau 3 de la chaîne de traçabilité).
STL	Acronyme	Standard Template Library. Ensemble des bibliothèques standard du langage C++ (conteneurs, algorithmes, flux, etc.).
UML	Acronyme	Unified Modeling Language. Langage de modélisation graphique standardisé utilisé pour représenter l'architecture logicielle.
<code>std::string</code>	Type C++	Type de la bibliothèque standard C++ pour la manipulation de chaînes de caractères.
<code>std::vector<T></code>	Type C++	Conteneur dynamique de la STL stockant des éléments de type T. Permet l'insertion en fin de tableau via <code>push_back()</code> .
<code>std::regex</code>	Type C++	Objet de la bibliothèque <code><regex></code> représentant une expression régulière compilée, utilisée pour filtrer du texte.
<code>std::ifstream</code>	Type C++	Flux de lecture de fichier (Input File Stream). Permet d'ouvrir et de lire un fichier texte ligne par ligne.
<code>std::getline()</code>	Fonction C++	Fonction standard C++ lisant une ligne complète depuis un flux d'entrée et la stockant dans une <code>std::string</code> .
<code>std::runtime_error</code>	Classe C++	Classe d'exception standard C++ utilisée comme classe de base pour les erreurs d'exécution.

Terme	Catégorie	Définition
push_back()	Méthode C++	Méthode de std::vector permettant d'ajouter un élément en fin de tableau dynamiquement.
std::thread	Classe C++	Classe de la bibliothèque <thread> (C++11) permettant de créer et gérer des threads d'exécution indépendants.
std::mutex	Classe C++	Mécanisme de synchronisation permettant de protéger l'accès concurrent à une ressource partagée entre plusieurs threads.
std::lock_guard	Classe C++	Garde de verrou RAII (C++11) qui acquiert un mutex à la construction et le libère automatiquement à la destruction.
join()	Méthode C++	Méthode de std::thread qui bloque le thread appelant jusqu'à ce que le thread ciblé ait terminé son exécution.
Encapsulation	Concept OOP	Principe de la programmation orientée objet consistant à restreindre l'accès direct aux attributs d'une classe en les déclarant privés, et à les exposer uniquement via des méthodes publiques contrôlées.
Héritage	Concept OOP	Mécanisme permettant à une classe (classe fille) de réutiliser les attributs et méthodes d'une autre classe (classe mère). Utilisé ici pour FileNotFoundException et FileFormatException héritant de std::runtime_error.
Exception	Concept C++	Mécanisme de gestion des erreurs en C++ permettant d'interrompre le flux normal d'exécution et de remonter l'erreur au module appelant via throw/catch.
Thread	Concept	Unité d'exécution légère permettant à un programme d'exécuter plusieurs tâches en parallèle. Utilisé dans le Module 2 pour séparer l'analyse de l'interface utilisateur.
Multithreading	Concept	Technique d'exécution simultanée de plusieurs threads au sein d'un même processus.
Expression régulière (Regex)	Concept	Séquence de caractères définissant un motif de recherche dans du texte. Utilisée dans le Module 1 pour identifier les identifiants d'exigences.

Terme	Catégorie	Définition
Initializer list	Concept C++	Syntaxe C++ permettant d'initialiser les attributs d'une classe directement lors de la construction de l'objet, avant l'exécution du corps du constructeur.
RAII	Concept C++	Resource Acquisition Is Initialization. Idiom C++ garantissant la libération automatique des ressources (fichiers, mémoire, verrous) à la fin de la portée d'un objet.
ExtracteurFichier	Classe (Module 1)	Classe principale du Module 1. Responsable de l'ouverture d'un fichier, de la lecture ligne par ligne et de l'extraction des identifiants d'exigences via une expression régulière.
FileNotFoundException	Classe (Module 1)	Exception levée lorsque le chemin fourni ne correspond à aucun fichier accessible (fichier inexistant, supprimé ou droits insuffisants).
FileFormatException	Classe (Module 1)	Exception levée lorsque le fichier fourni existe mais n'est pas dans un format textuel attendu.
MoteurTracabilite	Classe (Module 2)	Classe centrale du Module 2. Analyse les liens entre exigences SSS, SRS et SDD, détecte les incohérences et calcule les taux de couverture.
Exigence	Classe (Module 2)	Classe mère abstraite regroupant les attributs communs à toutes les exigences (identifiant, contenu, type, statut).
ExigenceSSS	Classe (Module 2)	Classe fille de Exigence représentant une exigence de niveau client (SSS).
ExigenceSRS	Classe (Module 2)	Classe fille de Exigence représentant une exigence de niveau projet (SRS).
ExigenceSDD	Classe (Module 2)	Classe fille de Exigence représentant une exigence de niveau conception (SDD).
RapportTracabilite	Structure (Module 2)	Structure de données transmise du Module 2 au Module 3, contenant les statuts des exigences, les liens brisés et les taux de couverture calculés.
MainWindow	Classe (Module 3)	Classe principale de l'interface graphique. Gère l'initialisation des composants, les zones d'affichage et les interactions utilisateur.

Terme	Catégorie	Définition
OngletRapport	Classe (Module 3)	Classe responsable de l'affichage de la vue bidimensionnelle.
VueGrapheTracabilite	Classe (Module 3)	Classe responsable de l'affichage arborescente des données du graphe de nœuds.
GestionnaireFiltresGUI	Classe (Module 3)	Classe gérant les filtres sur le document, par sous-document, sans modifier les données sous-jacentes.
Traçabilité	Terme métier	Capacité à relier un nœud à un niveau document, en fonction de l'exigence du niveau, garantissant la cohérence des données d'exigences.
Lien brisé	Terme métier	Référence dans un document vers un identifiant qui n'existe pas dans le document correspondant.
Taux de couverture	Terme métier	Indicateur quantitatif mesurant la proportion de données au niveau document par rapport au niveau inférieur.
Taux global	Terme métier	Indicateur synthétique calculé en multipliant le taux de couverture SRS→SDD, reflétant la qualité de la chaîne de traitement.
Identifiant d'exigence	Terme métier	Chaîne de caractères utilisée pour identifier sans ambiguïté une exigence dans un document. Format : EXIGENCE_CLIENT.
Frontière système	Terme métier	Limite logique entre le système et son environnement. Elle constitue la frontière du système de fichiers de l'application.
Parsing	Terme technique	Analyse syntaxique d'une chaîne de caractères pour identifier les éléments structuraux définies.
NoeudFichier	Struct (M3)	
LienFichier	Struct (M3)	
FiltreGUI	Struct (M3)	

Orpheline SRS	Terme métier
Orpheline SDD	Terme métier
TauxTracabiliteComplete	Terme métier
ConvertisseurExigences	Classe (M1)
LibreOffice (appel système)	Technique
Accueil	Classe (M3)

8 Index

L'index ci-dessous recense les termes, classes, méthodes et concepts clés du document, avec renvoi vers la ou les sections les mentionnant. Les entrées sont classées par ordre alphabétique.

C

Terme / Symbole	Section(s)
calculerTaux()	§ 4.5 — Méthode du MoteurTracabilite
cheminSource	§ 3.5.1 — Attribut d'ExtracteurFichier
Code couleur (vert / rouge / orange)	§ 5.1, § 6 — Interface graphique & Matrice

E

Terme / Symbole	Section(s)
Encapsulation	§ 3.4, § 4.4 — Interfaces et visibilité
Exigence (classe mère)	§ 4.5 — Classes du Module 2
ExigenceSSS	§ 4.5 — Classes du Module 2
ExigenceSRS	§ 4.5 — Classes du Module 2
ExigenceSDD	§ 4.5 — Classes du Module 2
Expression régulière (Regex)	§ 3.3, § 3.5.1 — Module 1
extraire()	§ 3.5.1 — Méthode publique d'ExtracteurFichier
ExtracteurFichier	§ 3.5.1 — Classe principale du Module 1

F

Terme / Symbole	Section(s)
MainWindow	§ 5.3.1 — Classe du Module 3
FormatException	§ 3.5.3 — Gestion des anomalies Module 1
FileNotFoundException	§ 3.5.3 — Gestion des anomalies Module 1
Filtrage (par document / statut / identifiant)	§ 5.1, § 6 — Interface graphique & Matrice
Frontière système	§ 3.1 — Description du Module 1

G

Terme / Symbole	Section(s)
GestionnaireFiltresGUI	§ 5.3.4 — Classe du Module 3
getLiensBrises()	§ 4.4 — Interface publique du MoteurTracabilite
getRapport()	§ 4.4 — Interface publique du MoteurTracabilite
getTauxGlobal()	§ 4.4 — Interface publique du MoteurTracabilite
getTauxSSS_SRS()	§ 4.4 — Interface publique du MoteurTracabilite
getTauxSRS_SDD()	§ 4.4 — Interface publique du MoteurTracabilite

H

Terme / Symbole	Section(s)
Héritage (std::runtime_error)	§ 3.5.3 — Exceptions personnalisées

I

Terme / Symbole	Section(s)
Identifiant d'exigence	§ 3.2, § 4.2, § 6 — Ensemble du document
Initializer list	§ 3.5.1 — Constructeur ExtracteurFichier

J

Terme / Symbole	Section(s)
join() (thread)	§ 4.3, § 4.5 — Multithreading Module 2

L

Terme / Symbole	Section(s)
liensBrises	§ 4.5 — Attribut de MoteurTracabilite
listeIdentifiants	§ 3.5.1 — Attribut d'ExtracteurFichier

M

Terme / Symbole	Section(s)
Matrice de traçabilité	§ 6 — Section dédiée
motifRegex	§ 3.5.1 — Attribut d'ExtracteurFichier
MoteurTracabilite	§ 4.5 — Classe principale du Module 2
Module 1 — Import & Parsing	§ 3 — Section dédiée
Module 2 — Moteur de Traçabilité	§ 4 — Section dédiée
Module 3 — Interface Graphique	§ 5 — Section dédiée
Multithreading	§ 4.3, § 4.5 — Module 2

P

Terme / Symbole	Section(s)
Parsing	§ 3.1 — Description du Module 1
push_back()	§ 3.5.1 — Algorithme de extraire()

R

Terme / Symbole	Section(s)
RAII	§ 3.5.1 — Mécanismes C++ utilisés
RapportTracabilite	§ 4.5, § 5.3 — Structure partagée Modules 2 & 3

S

Terme / Symbole	Section(s)
<code>std::mutex</code>	§ 4.5 – Synchronisation des threads
SDD	§ 2.2, § 4.2, § 6 – Ensemble du document
<code>setDonnees()</code>	§ 4.4 – Interface publique du MoteurTracabilite
SRS	§ 2.2, § 4.2, § 6 – Ensemble du document
SSS	§ 2.2, § 4.2, § 6 – Ensemble du document
<code>std::getline()</code>	§ 3.5.1 – Algorithme de extraire()
<code>std::ifstream</code>	§ 3.5.1 – Mécanismes C++ utilisés
<code>std::regex</code>	§ 3.3, § 3.5.1 – Module 1
<code>std::thread</code>	§ 4.3, § 4.5 – Module 2
<code>std::vector<std::string></code>	§ 3.5.1, § 4.5 – Modules 1 & 2

T

Terme / Symbole	Section(s)
Taux de couverture (SSS→SRS, SRS→SDD)	§ 4.2, § 4.5 – Module 2
Taux global de traçabilité	§ 4.2, § 4.5, § 5.1 – Modules 2 & 3
Thread d'analyse	§ 4.3 – Architecture multithreads Module 2
Traçabilité	§ 1.2, § 4 – Introduction & Module 2

V

Terme / Symbole	Section(s)
<code>VueGrapheTracabilite</code>	§ 5.3.3 – Classe du Module 3
<code>OngletRapport</code>	§ 5.3.2 – Classe du Module 3

9 Référence

Cette section liste l'ensemble des documents internes au projet et des ressources techniques externes qui ont servi de base à la rédaction de cette conception détaillée.

Documents internes au projet (Documents Applicables) :

- [REF-01] Cahier des Charges de l'outil de traçabilité — L2G1
- [REF-03] Cahier de Recette de l'outil de traçabilité — L2G1

Ressources techniques externes (Webographie / Bibliographie) :

- [REF-04] Documentation officielle du langage C++ (Standard C++11 et ultérieurs) : <https://en.cppreference.com/>
- [REF-05] Documentation de la bibliothèque zlib et minizip, disponible sur zlib.net.
- [REF-06] Documentation de la bibliothèque tinyxml2.
- [REF-07] Documentation Qt.

10 Suivi des modifications

Cette section recense l'ensemble des modifications apportées au document de la conception détaillée au fil des versions.

ID	Titre	Commentaires
6002	Modification schéma module 1	Restructuration visuelle pour améliorer la fluidité de lecture.
6002	Mise en forme avec un tableau de la partie 3.5	Transformation du bloc de texte en tableau comparatif. L'objectif est de faciliter la lecture rapide des données et de mettre en évidence les points clés de la section.
6003	Création tableau de version	Tracer l'historique des modifications.
6028	Ajout de méthodes dans la classe MainWindow	parcourirSSS/SRS/SDD(), lancerAnalyse(), logTerminal(), reinitialiser().
6029	Ajout de méthodes dans la classe OngletRapport	afficherRapportComplet(), exporterCSV(), exporterHTML(), exporterPDF().
6030	Ajout de méthodes dans la classe VueGrapheTracabilite	mousePressEvent(), wheelEvent(), appliquerFiltre().
6031	Ajout de la classe GestionnaireFiltresGUI	appliquerFiltres(), reinitialiser(), setListeIds(), getFiltreActuel().
6032	Ajout figure 4	Diagramme de flux utilisateur du module 3.
7402	Ajout formats .xlsx, .doc, .xls (Module 1)	Nouvelles méthodes : lireXlsx(), lireDoc(), lireXls(), chargerSharedStrings(), parserBlocsDocx(). Conversion .doc/.xls via LibreOffice.
7403	Ajout attribut fichierSource (Module 1)	Attribut fichierSource ajouté à la classe mère Exigence avec getter/setter. Propagé dans ConvertisseurExigences pour SSS, SRS et SDD.
7404	Refactorisation ConvertisseurExigences (Module 1)	Préfixes configurables par fichier (plusieurs valeurs séparées par virgule). Méthodes splitVirgule(), detecterType(), estTypeConnu().
7405	Séparation orphelines / liens brisés (Module 2)	Vecteur orphelines distinct de liensBrises. Getter getOrphelines() ajouté. Déduplication automatique entre les deux listes.
7406	Ajout taux de traçabilité complète (Module 2)	Attribut tauxTracabiliteComplete dans MoteurTracabilite et RapportTracabilite. Calcul dans calculerTaux().
7407	ThreadManager : atomic<bool> sans mutex (Module 2)	analyseTerminee déclaré en std::atomic<bool>. Destructeur joint le thread automatiquement (~ThreadManager).
7408	Ajout classe Accueil — splash screen (Module 3)	Nouvelle fenêtre de démarrage Qt avec barre de progression animée (QTimer 25ms) et signal demarrerClique() connecté à MainWindow.
7409	MainWindow : import multi-fichiers (Module 3)	Sélection multiple SSS/SRS/SDD. Préfixes configurables par fichier via QMap<QString, QLineEdit*>. Méthode creerChampsPrefixes().
7410	Ajout classe OngletRapport (Module 3)	Remplace VueMatriceTracabilite. Matrices 3 colonnes SSS×SRS et SRS×SDD. Panneaux liens brisés (orange) et orphelines (violet). Exports CSV, HTML interactif, PDF.
7411	VueGrapheTracabilite v5 (Module 3)	Nœuds par fichier (NoeudFichier). Couleurs fixes par type. Vue directe/indirecte. Badge pourcentage sur les liens.
7412	GestionnaireFiltresGUI v1.1 (Module 3)	Mode graphe (cache statut/état). Filtre orpheline (99) et lien brisé (98). Autocomplétion via QCompleter.

7413	Redessin des figures 1 à 6	Tous les diagrammes (architecture, flux import, threads, activité Module 2, UML classes, flux utilisateur) mis à jour selon les classes du code.
------	----------------------------	--

Version	Date	Émetteur	Valideur	Raisons
A1	15.03.26	Emilie Huang	Jérémy Meynadier	
A2	03.05.2026	Lamia Arrahmane	Jérémy Meynadier	